

Insurance

Homework #4

Naomi Buell Richie Rivera Alexander Simon Nana Frimpong
Said Naqwe

2026-04-26

1 Introduction

We model two outcomes in a auto insurance outcome variables: (1) whether the person was in a car crash (binary) and (2) the amount of money it will cost if the person does crash their car (continuous). Our objective is to build multiple linear regression and binary logistic regression models on the training data to predict the probability that a person will crash their car and also predict the cost of the crash if it occurs.

2 Data Exploration and Cleaning

```
# Load the training and evaluation datasets
insurance_train <- read_csv("./data/insurance_training_data.csv")
insurance_eval <- read_csv("./data/insurance-evaluation-data.csv")

# Count rows and columns
nrows <- insurance_train |> nrow()
ncols <- insurance_train |> length()

# Get missingness
perc_complete <- round(insurance_train |> n_complete() / nrows / ncols * 100, 2)
```

The insurance training data set contains 8161 rows and 26 features, and is 98.87 percent complete. We will need to address the missingness in the data preparation section.

There are several variables in the dataset coded as characters. There are several currency variables (INCOME, HOME_VAL, BLUEBOOK, OLDCLAIM) that should be changed to numeric variables. The rest of the character variables should be encoded as dummy variables for modeling in the following chunk.

We also rename some variables for clarity. We also remove the `index` variable as it is not a predictor and will not be used in modeling.

```
# Convert currency variables to numeric by removing "$" and converting to numeric
currency_vars <- c("INCOME", "HOME_VAL", "BLUEBOOK", "OLDCLAIM")

# Character columns to one-hot encode (exclude currency columns)
char_vars <- insurance_train |>
  select(where(is.character)) |>
  names() |>
  setdiff(currency_vars)

insurance_train_dummies <- insurance_train |>
  mutate(
    # Convert currency variables to numeric
    across(all_of(currency_vars), parse_number)
  ) |>
  fastDummies::dummy_cols(
    select_columns = char_vars,
    remove_selected_columns = TRUE,
    remove_first_dummy = TRUE
  ) |>
  clean_names() |>
  rename(
    female = sex_z_f,
    education_phd = education_ph_d,
    education_high_school = education_z_high_school,
    car_type_suv = car_type_z_suv,
    job_blue_collar = job_z_blue_collar,
    urbanicity_highly_rural = urbanicity_z_highly_rural_rural
  ) |>
  select(-index)

insurance_eval_dummies <- insurance_eval |>
  mutate(
    across(all_of(currency_vars), parse_number)
  ) |>
  fastDummies::dummy_cols(
    select_columns = char_vars,
    remove_selected_columns = TRUE,
```

```

    remove_first_dummy = TRUE
  ) |>
  clean_names() |>
  rename(
    female = sex_z_f,
    education_phd = education_ph_d,
    education_high_school = education_z_high_school,
    car_type_suv = car_type_z_suv,
    job_blue-collar = job_z_blue-collar,
    urbanicity_highly_rural = urbanicity_z_highly_rural_rural
  ) |>
  select(-index)

```

In previewing summary statistics for all variables, some of the minimum values appear to be in error. For example, `car_age` has a negative value, and there are a high number of 0 values in `car_age`, `home_val`, which may be data entry errors—we will replace them missing values.

```

insurance_train_dummies |> summary()

insurance_train_dummies <- insurance_train_dummies |>
  mutate(
    car_age = ifelse(car_age <= 0, NA, car_age),
    home_val = ifelse(home_val == 0, NA, home_val)
  )

```

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness						
	rate	Mean	Min	P25	P50	P75	Max
target_flag	1.00	0.260	0	0	0	1	1
target_amt	1.00	1500.000	0	0	0	1000	110000
kidsdriv	1.00	0.170	0	0	0	0	4
age	1.00	45.000	16	39	45	51	81
homekids	1.00	0.720	0	0	0	1	5
yoj	0.94	10.000	0	9	11	13	23
income	0.95	62000.000	0	28000	54000	86000	370000
home_val	0.66	220000.000	50000	150000	210000	270000	890000
travtime	1.00	33.000	5	22	33	44	140
bluebook	1.00	16000.000	1500	9300	14000	21000	70000
tif	1.00	5.400	1	1	4	7	25
oldclaim	1.00	4000.000	0	0	0	4600	57000

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness						
	rate	Mean	Min	P25	P50	P75	Max
clm_freq	1.00	0.800	0	0	0	2	5
mvr_pts	1.00	1.700	0	0	1	3	13
car_age	0.94	8.300	1	1	8	12	28
parent1_yes	1.00	0.130	0	0	0	0	1
mstatus_z_no	1.00	0.400	0	0	0	1	1
female	1.00	0.540	0	0	1	1	1
education_bachelors	1.00	0.270	0	0	0	1	1
education_masters	1.00	0.200	0	0	0	0	1
education_phd	1.00	0.089	0	0	0	0	1
education_high_school	1.00	0.290	0	0	0	1	1
job_doctor	0.94	0.032	0	0	0	0	1
job_home_maker	0.94	0.084	0	0	0	0	1
job_lawyer	0.94	0.110	0	0	0	0	1
job_manager	0.94	0.130	0	0	0	0	1
job_professional	0.94	0.150	0	0	0	0	1
job_student	0.94	0.093	0	0	0	0	1
job_blue-collar	0.94	0.240	0	0	0	0	1
job_na	1.00	0.064	0	0	0	0	1
car_use_private	1.00	0.630	0	0	1	1	1
car_type_panel-truck	1.00	0.083	0	0	0	0	1
car_type_pickup	1.00	0.170	0	0	0	0	1
car_type_sports-car	1.00	0.110	0	0	0	0	1
car_type_van	1.00	0.092	0	0	0	0	1
car_type_suv	1.00	0.280	0	0	0	1	1
red-car_yes	1.00	0.290	0	0	0	1	1
revoked_yes	1.00	0.120	0	0	0	0	1
urbanicity_highly-urban	1.00	0.200	0	0	0	0	1

Table 1 presents descriptive characteristics of the training data. The target variables are `target_flag`, which represents whether the person was in a car crash (1) or not (0), and `target_amt`, which represents the cost of the crash if it occurs. The other variables represent various information on auto insurance for different individuals.

```
# Get correlation matrix
insurance_train_cor <- insurance_train_dummies |>
  select(where(is.numeric)) |>
  cor(use = "pairwise.complete.obs")

insurance_train_cor_plot <- insurance_train_cor
insurance_train_cor_plot[abs(insurance_train_cor_plot) < 0.2] <- NA

corrplot(insurance_train_cor_plot, method = "circle", type = "upper", sig.level = 0.05, insig = 0.05)

most_corr_w_target_flag <- insurance_train_cor |>
  as.data.frame() |>
  rownames_to_column(var = "Variable") |>
  filter(Variable != "target_flag", Variable != "target_amt") |>
  transmute(
    Variable,
    Correlation = target_flag,
    Abs_Correlation = abs(target_flag)
  ) |>
  arrange(desc(Abs_Correlation)) |>
  filter(Abs_Correlation > 0.2) |> pull(Variable)

most_corr_w_target_amt <- insurance_train_cor |>
  as.data.frame() |>
  rownames_to_column(var = "Variable") |>
  filter(Variable != "target_flag", Variable != "target_amt") |>
  transmute(
    Variable,
    Correlation = target_amt,
    Abs_Correlation = abs(target_amt)
  ) |>
  arrange(desc(Abs_Correlation)) |>
  filter(Abs_Correlation > 0.2) |>
  pull(Variable)
```

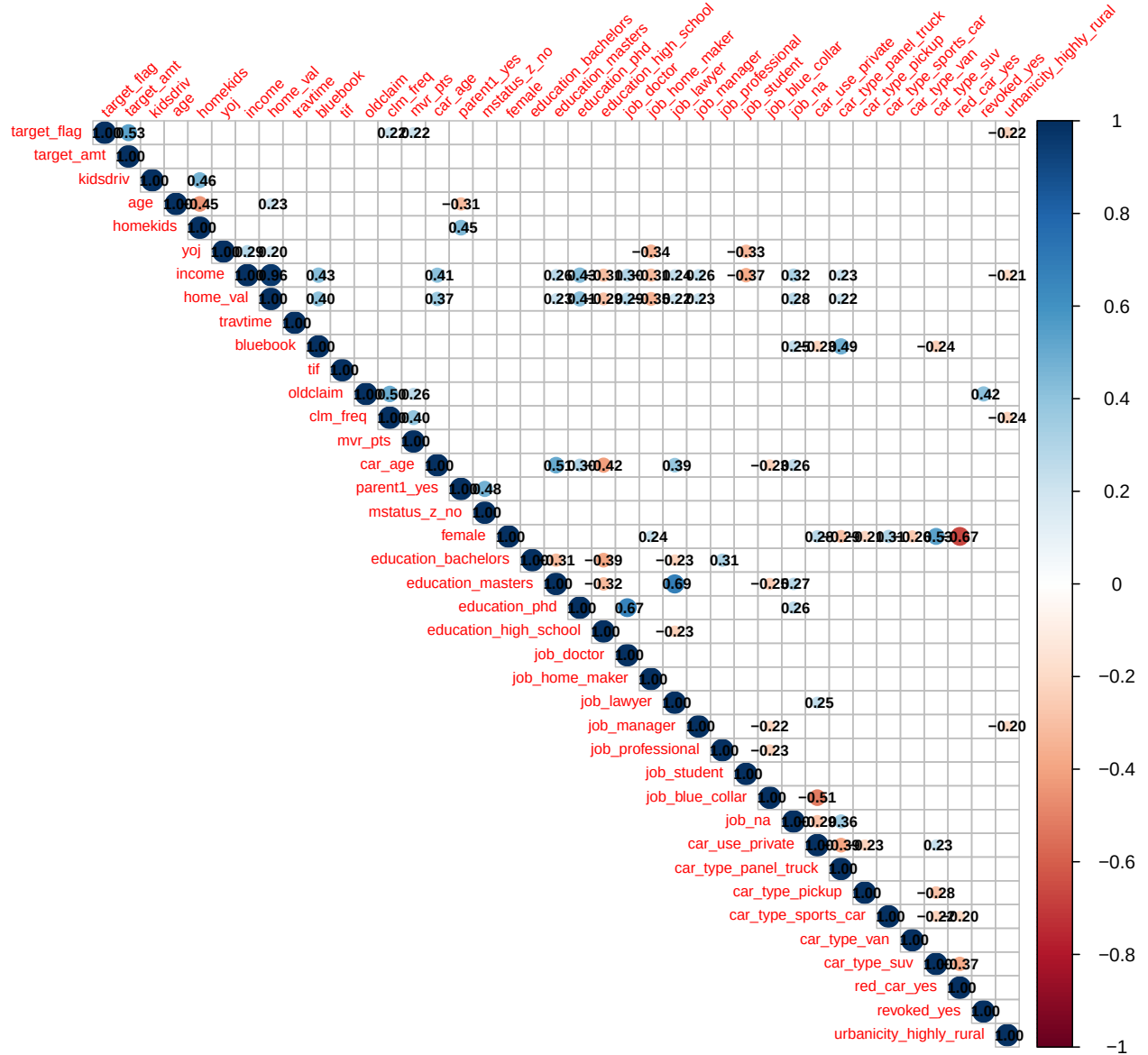


Figure 1: Correlation matrix of all variables in the training dataset. Note: Correlations with absolute value less than 0.2 are suppressed.

The correlation between all variables are shown Figure 1. The variables most correlated with **target_flag**, whether there was a car crash, (besides the amount the crash costed) are **urbanicity_highly_rural**, **mvr_pts**, **clm_freq**. This means that people who work or live in a more rural area are less likely to be in a crash, and people who have more motor vehicle record points or have more claims in the past 5 years are more likely to be in a crash. There were no predictor variables strongly correlated with **target_amt**, the cost if the crash occurs.

Lastly, for continuous variables, we also looked at the distribution of each variable to check for skewness and outliers.

```
# Keep only continuous
continuous_vars <- insurance_train_dummies |>
  select(where(is.numeric)) |>
  select(where(~ dplyr::n_distinct(stats::na.omit(.x)) > 10)) |>
  names()

insurance_train_dummies |>
  select(all_of(continuous_vars)) |>
  pivot_longer(cols = everything(), names_to = "variable", values_to = "value") |>
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ variable, scales = "free", ncol = 4) +
  theme_minimal() +
  theme(panel.grid = element_blank()) +
  labs(x = NULL, y = "Count")
```

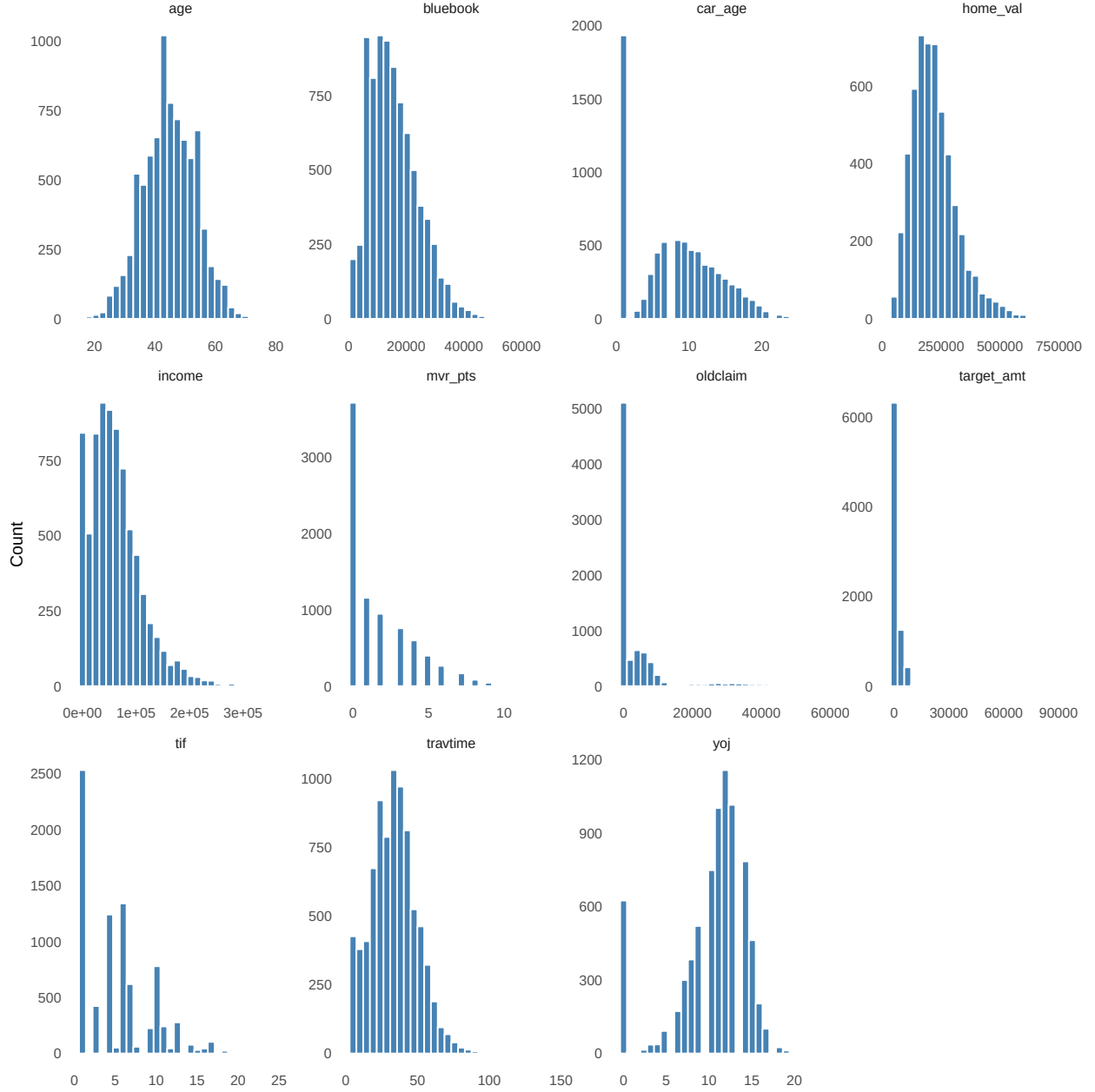


Figure 2: Distribution of continuous variables in the training dataset.

There are many 1-year old cars in the dataset, showing a high spike at 1 in the `car_age` variable. Dollar amounts such as `bluebook`, `home_val`, `income`, and `target_amt` are right-skewed, which is expected for financial variables. The same goes for travel time (`travtime`). Most people have no motor vehicle record points (`mvr_pts`) or claims in the past 5 years (`oldclaim`), which is expected for a general population. Years on the job (`yoj`) is almost normally distributed, but has a spike at 0, which may represent people who are unemployed or have been on the job for less than a year.

3 Data Preparation

For the crime data used in Homework 3, the first step in data preparation was to check whether any variables contained missing values. This crime data set does not contain missing values in either the training or evaluation files. Therefore, no imputation or missingness flag variables were needed.

Even though missing-value treatment was unnecessary, we still prepared the data for binary logistic regression by creating transformed and derived predictors. Several predictors are right-skewed, especially `zn`, `dis`, `rad`, `tax`, and `lstat`. For these variables, log transformations were created using `log1p()`, which reduces skewness while safely handling zero values. These transformed variables may improve model fit by making relationships with the log-odds of the response more stable.

We also created a small set of engineered variables that may be useful in later models. First, `nox_indus` captures the combined effect of air pollution concentration and industrial land use. Second, `rm_lstat_ratio` measures the balance between average rooms per dwelling and lower-status population, combining a housing-quality measure with a socioeconomic measure. Third, `tax_rad_ratio` captures the relationship between tax burden and highway accessibility. These new variables were created because logistic regression can benefit from transformed and combined predictors when the original variables may not fully capture the structure related to high-crime neighborhoods.

Finally, the cleaned training and evaluation data sets were stored as `crime_clean` and `crime_eval_clean` so the later modeling section can use them directly.

```
# Check missing values in training and evaluation data
missing_train <- crime_train |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing")

missing_eval <- crime_eval |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing")

missing_train
missing_eval

# Start with copies of the original datasets
crime_clean <- crime_train
crime_eval_clean <- crime_eval

# Create transformed and engineered predictors
crime_clean <- crime_clean |>
```

```
mutate(
  # Log transforms for right-skewed predictors
  log_zn = log1p(zn),
  log_dis = log1p(dis),
  log_rad = log1p(rad),
  log_tax = log1p(tax),
  log_lstat = log1p(lstat),

  # Engineered variables
  nox_indus = nox * indus,
  rm_lstat_ratio = rm / lstat,
  tax_rad_ratio = tax / rad
)

crime_eval_clean <- crime_eval_clean |>
mutate(
  # Log transforms for right-skewed predictors
  log_zn = log1p(zn),
  log_dis = log1p(dis),
  log_rad = log1p(rad),
  log_tax = log1p(tax),
  log_lstat = log1p(lstat),

  # Engineered variables
  nox_indus = nox * indus,
  rm_lstat_ratio = rm / lstat,
  tax_rad_ratio = tax / rad
)

# Verify dimensions after preparation
dim(crime_clean)
dim(crime_eval_clean)

# Verify that no missing values remain after transformations
crime_clean |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing") |>
  filter(Missing > 0)
```

```
crime_eval_clean |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing") |>
  filter(Missing > 0)
```

3.0.1 Linearity Assumption Summary

The results above indicate that the assumption of linearity is satisfied for all independent variables. However, several were flagged as being unlikely to have predictive value in a multivariate linear model.

3.1 Data Splits

First, we will split our dataset into a training and testing set. We will use the training set to build our models and the testing set to evaluate their performance. The training set will be used to fit the models, while the testing set will be used to assess how well the models generalize to new data.

```
set.seed(19940211)
train_indices <- sample(nrow(crime_clean), size = 0.8 * nrow(crime_clean))
crime_train_split <- crime_clean[train_indices, ]
crime_test_split <- crime_clean[-train_indices, ]
```

4 Check assumptions for binary logistic regression

1. Binary Outcome

```
# Confirming that the target is binary
crime_clean |>
  count(target) |>
  mutate(prop = n / sum(n))
```

The response variable “target” is confirmed to be a binary, therefore it takes two values (0: crime rate below the median) and (1: crime rate above the median). In the 466 neighborhoods in training data set 237 (50.9%) fall in the class of low crime and 228 (49.1)% are in the high crime neighborhood. There is no class imbalance, due to the near equal split meaning there is unlikely to be bias towards one class.

2. Independence of Observations

Each record in the dataset represents a distinct neighborhood, and there is no repeated-measures or clustered structure in the data. Therefore, the independence of observations assumption is satisfied by design and requires no further adjustment.

3. No Multicollinearity (VIF Check)

```
library(car)

# Fit a reference logistic model with original predictors
model_ref <- glm(
  target ~ zn + indus + chas + nox + rm + age + dis + rad + tax + ptratio
  + lstat + medv,
  data = crime_clean, family = binomial
)

# Compute VIF
vif_vals <- vif(model_ref) |>
  as.data.frame() |>
  rownames_to_column("Variable") |>
  rename(VIF = 2) |>
  arrange(desc(VIF))

vif_vals |> knitr::kable(caption = "Variance Inflation Factors for Reference Model")
```

The VIF values is calculated for all predictors to check for multicollinearity. A VIF above 10 is considered to be severe, while between 5 and 10 should warrant caution. None of the variable exceed a threshold of 10, indicating multicollinearity is not present. The highest VIF belongs to medv (median home value, VIF = 8.12), followed by rm (average rooms per dwelling, VIF = 5.81). These two variables are moderate and do not need require any removal. The remaining predictors fall below 5 and are acceptable levels of multicollinearity. There was strong pairwise correlation between “rad” and “tax” in the exploration section, however it is confirmed that multicollinearity does not exist in the dataset.

4. Linearity of Log-Odds (Box-Tidwell Test)

```
# Box-Tidwell test: add interaction of each continuous predictor with its log
# to test whether the log-odds relationship is linear
continuous_vars <- c(
  "zn", "indus", "nox", "rm", "age", "dis", "rad", "tax",
  "ptratio", "lstat", "medv"
)

# Create interaction terms x * log(x) for each continuous predictor
# (use log1p to handle zeros)
crime_bt <- crime_clean |>
```

```

    mutate(across(all_of(continuous_vars), ~ . * log1p(.), .names = "{.col}_logx"))

bt_formula <- as.formula(paste(
  "target ~",
  paste(continuous_vars, collapse = " + "),
  "+",
  paste(paste0(continuous_vars, "_logx"), collapse = " + ")
))

model_bt <- glm(bt_formula, data = crime_bt, family = binomial)

# Extract p-values for the interaction terms only
bt_results <- summary(model_bt)$coefficients |>
  as.data.frame() |>
  rownames_to_column("Term") |>
  filter(str_detect(Term, "_logx")) |>
  mutate(
    Variable = str_remove(Term, "_logx"),
    Significant = `Pr(>|z|)` < 0.05
  ) |>
  select(Variable, Estimate, `Pr(>|z|)`, Significant) |>
  mutate(across(where(is.numeric), ~ signif(., 3)))

bt_results |> knitr::kable(caption = "Box-Tidwell Test for Linearity of Log-Odds")

```

The Box-Tidwell test was used to assess whether each continuous predictor has a linear relationship with the log-odds of the outcome. A p-value greater than 0.05 violate the assumption. There are three variables that were flagged as significant:rm ($p = 0.003$), dis ($p < 0.001$), and tax ($p < 0.001$). This indicates that these predictors do not have strict linear relationship with log-odds of high crime. Log transformation were applied to these variables in the data preparation section in order to be used in the model building. The remaining eight predictors did not have any evidence of violating the linearity assumption.

5 Build models

We compared three models. Our first model included all the variables in the cleaned training dataset. The second model aimed to reduce multicollinearity by omitting variables that had correlation coefficients $|r| > 0.7$ (see Data Exploration section). Finally, we generated the third model using backward stepwise selection from a full model that included transformed variables and newly created

variables (see Data Preparation section).

5.1 Model 1. All untransformed variables

As shown below, the most significant predictors were `nox` and `rad` ($P < 0.001$). In addition, `age` and `ptratio` were significant at the $P < 0.01$ level, and `dis` and `tax` were significant at the $P < 0.05$ level. The AIC for this model is `r\ AIC(model1)`.

```
model1 <- glm(  
  formula = target ~ zn + indus + chas + nox + rm + age + dis + rad + tax +  
    ptratio + lstat + medv,  
  family = "binomial",  
  data = crime_train_split  
)  
  
summary(model1)
```

5.2 Model 2. Reduce multicollinearity

In the Data Exploration section, we showed that `rad` and `tax` were strongly correlated ($r=0.91$), which suggests that only one of these variables is needed in Model 1 (we chose to retain `rad` as it was more highly significant). Similarly, `medv` was shown to be potentially multicollinear with `lstat` ($r=0.74$) and `rm` ($r=0.71$), so we removed the latter variables.

In addition, the correlation between `nox` (predictor) and `target` (response) suggests that `nox` is an important predictor, so we removed its potential collinear pairs `age` and `dis`. Although `nox` was moderately correlated with `indus` ($r=0.76$), we retained `indus` because removal of both `indus` and `tax` (highly correlated with `rad`) would eliminate a potentially informative predictor.

These changes resulted in a model with AIC `r\ AIC(model2)`.

```
model2 <- glm(  
  formula = target ~ zn + indus + chas + nox + rad + ptratio + medv,  
  family = "binomial",  
  data = crime_train_split  
)  
  
summary(model2)
```

5.3 Model 3. Backward stepwise variable selection

For this model, we performed backward stepwise selection from the set of variables including transformed and newly created variables.

The best model had `nox`, `tax_rad_ratio`, `ptratio`, and `age` as the most significant predictors ($P < 0.001$). In addition, `log_dis` and `rm_lstat_ratio` were significant at the $P < 0.05$ level. The AIC of this model was `r\ AIC(model3)`, which was lower than that of Models 1 and 2.

```
# Full model includes all variables
model_full <- glm(
  formula = target ~ log_zn + indus + chas + nox + rm + age +
    log_dis + log_rad + log_tax + ptratio + log_lstat + medv +
    nox_indus + rm_lstat_ratio + tax_rad_ratio,
  family = "binomial",
  data = crime_train_split
)

# Select variables
model3 <- step(model_full, direction = "backward")

summary(model3)
```

Of these three models, model 3 has the lowest AIC. In the next section, we will evaluate the performance of these models to more determine which one is best for predicting the `target` variable.

6 Select Models

```
# Creating a custom helper function to extract all the required metrics
# explicitly evaluating on the provided training dataset.
get_logistic_metrics <- function(model, model_name, data, target_var = "target") {
  # 1. Get predicted probabilities and classes
  probs <- predict(model, newdata = data, type = "response")
  preds <- factor(ifelse(probs > 0.5, 1, 0), levels = c(0, 1))
  actuals <- factor(data[[target_var]], levels = c(0, 1))

  # 2. Generate Confusion Matrix and derived metrics
  cm <- confusionMatrix(preds, actuals, positive = "1")

  # 3. Calculate AUC
  roc_obj <- roc(data[[target_var]], probs, quiet = TRUE)
  auc_val <- as.numeric(pROC::auc(roc_obj))

  # 4. Return results as a tibble
  tibble(
```

```

    Model = model_name,
    Deviance = model$deviance,
    Accuracy = cm$overall["Accuracy"],
    Error_Rate = 1 - cm$overall["Accuracy"],
    Precision = cm$byClass["Precision"],
    Sensitivity = cm$byClass["Sensitivity"], # Also Recall
    Specificity = cm$byClass["Specificity"],
    F1 = cm$byClass["F1"],
    AUC = auc_val,
    AIC = AIC(model),
    BIC = BIC(model)
  )
}

models_list <- list(
  "All Untransformed Variables" = model1,
  "Reducing Multicollinearity" = model2,
  "Full Model with Backward Selection" = model_full
)

# Apply the function to all three models, explicitly passing the training split
model_comparison <- imap_dfr(models_list, ~ {
  get_logistic_metrics(.x, .y, crime_test_split)
}) |>
  arrange(`Deviance`)

# Transpose the data and convert to a data frame
model_t <- as.data.frame(t(model_comparison))

# 3. Display the table
knitr::kable(
  model_t,
  digits = 4,
  caption = "Evaluation of Candidate Models on Training Dataset"
)

best_model_info <- model_comparison |> slice(1)

best_model_obj <- models_list[[best_model_info$Model]]

```


For the logistic models, we decided to use the deviance as our primary criterion on the training dataset, which was attained by the `best_model_info$Model`. We've chosen deviance as our primary metric because it directly measures the goodness of fit for logistic regression models, with lower deviance indicating a better fit to the training data. Given our model results, each model had a fairly similar accuracy with our `best_model_info$Model` having the highest accuracy of `round(best_model_info$Accuracy, 3)`. It also had the lowest error rate of `round(best_model_info$Error_Rate, 3)` and was comparable with the other top models in sensitivity and specificity. Overall, this model had the best balance of fit and predictive performance on the training data, which is why we selected it for further evaluation on the test set and for generating predictions on the evaluation set.

Given that our target variable was fairly balanced (49% of all observations had a target value of 1), and our accuracy, Precision, and Sensitivity show that we greatly outperform a random classifier, we are confident that our model is performing well.

```
# Display the summary of the selected Stepwise model to interpret coefficients and significance
summary(best_model_obj)
```

To make inferences from our top performing model, we can look at the significance of the predictors. The logistic regression model identifies several key environmental and structural factors that significantly predict whether a suburb's crime rate will exceed the median. The most dominant predictor is the concentration of nitrogen oxides (nox), where higher levels are strongly associated with a higher probability of the area being high-crime ($z=4.670, p<0.001$). Additionally, the pupil-teacher ratio (ptratio) and the proportion of older homes (age) built before 1940 are both significant positive predictors, suggesting that areas with crowded schools and aging infrastructure are more likely to see elevated crime rates. The model also highlights that as the distance to employment centers (log_dis) increases, so does the likelihood of high crime, along with a significant effect from the ratio of rooms to lower-status population (rm_lstat_ratio). Interestingly, many traditional economic indicators, such as median home values (medv), property tax rates (log_tax), and zoning for large lots (log_zn), did not reach statistical significance, indicating that in this specific dataset, industrial and demographic markers are more reliable indicators of crime than general wealth or proximity to the Charles River.

7 Results

```
# Generate predictions on the evaluation dataset using the selected model
eval_predictions <- predict(best_model_obj, newdata = crime_eval_clean, type = "response")

crime_eval_results <- crime_eval_clean |>
  mutate(
    # If probability > 0.5, it's a 1 (high crime), otherwise 0
```

```
PREDICTED_TARGET = ifelse(eval_predictions > 0.5, 1, 0)
)

# Write predictions to a csv
write_csv(crime_eval_results, "crime_evaluation_predictions.csv")

# Display the first 10 predictions in the rendered PDF
knitr::kable(
  head(crime_eval_results |> select(PREDICTED_TARGET), 10),
  caption = "Excerpts of Stepwise Model Win Predictions for Evaluation Set"
)
```

Given our evaluation set and our stepwise model, we are able to create predictions on the evaluation set on which neighborhoods are classified as high-crime or low-crime. The first 10 predictions are shown in the table above. We write these predictions to a CSV file for submission.

8 Conclusion

In conclusion, our `best_model_info$Model` model, which was selected based on deviance and other performance metrics, demonstrates strong predictive capabilities for classifying neighborhoods as high-crime or low-crime based on a variety of environmental and structural factors. The significant predictors identified in the model provide valuable insights into the underlying drivers of crime in urban areas, highlighting the importance of air pollution, school crowding, aging infrastructure, and proximity to employment centers. These findings can inform policymakers and community leaders in their efforts to target interventions and allocate resources effectively to reduce crime rates. Future work could explore additional modeling techniques, such as regularization or ensemble methods, to further enhance predictive performance and interpretability.